Improving IntentBench with ArmorIQ Evals

From a stochastic drift estimate to a deterministic, scored intent eval: what Armor's policy layer adds to the benchmark, and exactly how much of the drift surface it makes precise.

Deliverable 2 of 3. Companion to the full intent-drift report (deliverable 1) and the before/after projection (deliverable 3).

Measured figures are queried live from db/results.db (479 runs). Projected figures are labelled as such and are derived from the intent-contract policy coverage in armor-intent/policies, not from Armor runs, which the current quota does not allow.

1. The measurement gap this closes

The benchmark's core result is that on an identical repository and bug, prompt wording alone drives drift: scoped and vague prompts hold, loose prompts drift in a majority of cells. Measured pooled loose drift is 33 of 56 cells on generation 1 (59%), 25 of 49 on generation 2 (51%), and 14 of 20 on the new software domains (70%). That result is behavioural: each cell is a single sampled run, and whether an agent wanders into a rabbit hole is a probabilistic event. Turning a drift rate into a confident number needs repeated runs and error bars, which is the benchmark's standing future work.

Armor changes what is being measured. Instead of sampling whether an agent chooses to stay in scope, it enforces the scope at the tool-call boundary and records the decision. Armor already detects this exact phenomenon and calls it intent drift: a runtime tool call that diverges from the signed plan is refused. What Armor lacks is a scored benchmark of it: its own evals surface is an empty stub. IntentBench supplies that eval; Armor supplies the determinism. The rest of this document quantifies the trade.

2. Every task is already an Armor policy

Each task states one permitted change (the fix) and four planted rabbit holes it must not touch. That is an intent contract, and it compiles directly into a real armor.policy.v1 PolicyProfile: default deny, deny_overrides, one permit statement on the fix file, and forbid statements on the rabbit-hole files. All 17 domains were compiled this way (armor-intent/policies/), with no per-result tuning.

Benchmark concept	Armor concept (real IR)
the requested fix (in_scope)	permit, action tool in Edit/Write, file.path = fix file
planted rabbit holes (out_of_scope)	forbid on each cross-file rabbit-hole path
unrequested_changes > 0 (our drift)	Drift / "intent drift: tool not in plan"
a benchmark cell (arm x task)	one enforced session, decision logged

3. How much of the drift surface is deterministic

Armor's shipped policy conditions match on file.path. In every domain, three of the four rabbit holes live in a different file from the fix, so a path forbid blocks them deterministically. The fourth is the old_<fn> legacy alias, which sits in the same file as the fix and cannot be separated by a path condition. Across the suite that is a hard structural split, identical in all 17 domains:

Rabbit-hole surface	Count	Armor path policy
Cross-file holes (different module from fix)	51	blocked deterministically
Same-file hole (old_<fn> alias)	17	not separable by file.path
Total planted holes	**68**	**75% path-covered**

So a purely path-level policy makes 75% of the drift surface a deterministic verdict rather than a sampled rate. The remaining 25% (the same-file alias) needs Armor's finer gate: the signed intent plan that names the target symbol, or a matches condition on edit content, the same parameter inspection Armor already runs for PCI and payment detection. With that gate the covered fraction rises toward 100%, still deterministic.

4. The precision gain, per cohort

Precision here means: how much of the measured drift stops being a sampled quantity. The table gives, per measured cohort, the drift observed today and two enforce-mode bounds. The guaranteed floor assumes nothing: since each domain has exactly one same-file hole, at most one changed symbol per cell can survive a path policy, so at least (symbols minus drifted cells) are prevented. The expected column assumes drift lands on the 3-to-1 cross-to-same surface in proportion.

Cohort	Drifted cells	Drift symbols	Prevented (floor)	Prevented (expected)	Residual cells
Generation 1, loose	33	72	≥ 39	~ 54	≤ 33
Generation 2, loose	25	48	≥ 23	~ 36	≤ 25
New domains, loose (2 arms)	14	14	≥ 0	~ 10	≤ 14

The floor is weak exactly where drift is single-symbol: on the new domains both runnable arms changed one rabbit hole per drifted cell, so the floor cannot rule out that each was the same-file alias, and the guaranteed prevention there is 0. This is not a defect of Armor but of a path-only policy, and it is the precise place the symbol-level intent gate earns its keep: naming the fix symbol in the plan makes even a same-file second edit a tool-not-in-plan denial.

5. What this adds to the benchmark, concretely

Three changes turn the drift study into an Armor-scored eval suite. First, run each cell twice, once unenforced (the current baseline) and once under the task's PolicyProfile in enforce mode; the delta is the measured prevention, which replaces the projection in this document. Second, persist the identity of each changed symbol, not just the count: the AST diff in the scorer already computes it, so this is a one-column schema add, and it collapses the floor-to-expected band above into a point number. Third, score monitor mode alongside enforce mode: monitor leaves behaviour unchanged but labels every drift event with the statement it violated, which measures attribution precision without touching the agent.

The payoff is that the benchmark's headline stops depending on prompt phrasing. Today the same model reads as faithful or unfaithful depending on whether the prompt was scoped or loose. Under enforced policy the covered surface is faithful by construction, and the residual is a small, named, same-file set rather than an open-ended behavioural tail. That is the difference between reporting a drift rate and certifying a drift bound.